

Automating Dockerfile Refactoring to Multi-stage Builds

DONGJIN CHEN, Nanjing University of Aeronautics and Astronautics, China

WENHUA YANG*, Nanjing University of Aeronautics and Astronautics, China

MINXUE PAN, Nanjing University, China

YU ZHOU, Nanjing University of Aeronautics and Astronautics, China

Containerization has become a cornerstone of modern software deployment, yet many projects still ship single-stage Dockerfiles that bundle compilers, build tools, and temporary files into production images, thereby hurting performance and security. Multi-stage builds are recommended, yet uptake appears uneven, plausibly because refactoring legacy Dockerfiles demands nontrivial reasoning about build lifecycles and dependency separation. This paper presents STAGECRAFT, an automated refactoring approach that converts single-stage Dockerfiles into optimized multi-stage builds. STAGECRAFT first performs static analysis to identify the technology stack and to infer build-time and runtime dependencies. It then applies a lightweight gate that estimates the refactoring benefit from a composite of image bloat, structural inefficiency, and security risk, proceeding only when refactoring is warranted. Finally, it synthesizes a multi-stage Dockerfile that isolates build tooling, copies only the artifacts needed at runtime, and applies production hardening. Evaluated on 521 real-world single-stage projects, STAGECRAFT successfully produced working multi-stage Dockerfiles for 60.3% of targets. The resulting images were, on average, 52.2% smaller and contained 50.0% fewer high-risk vulnerabilities than the originals, outperforming baselines. STAGECRAFT lowers the barrier to multi-stage adoption at scale, yielding leaner images with a reduced attack surface and improved maintainability. We release the tool, its knowledge assets, and the evaluation dataset to support reproducibility and future research.

CCS Concepts: • **Software and its engineering** → **Software configuration management and version control systems**; **Software maintenance tools**; *Software development techniques*.

Additional Key Words and Phrases: Dockerfile, Refactoring, Docker, Multi-stage build

ACM Reference Format:

Dongjin Chen, Wenhua Yang, Minxue Pan, and Yu Zhou. 2026. Automating Dockerfile Refactoring to Multi-stage Builds. *Proc. ACM Softw. Eng.* 3, FSE, Article FSE070 (July 2026), 22 pages. <https://doi.org/10.1145/3797098>

1 Introduction

Containerization, with Docker as a leading technology, has reshaped software delivery by providing consistent, scalable, and reproducible environments. The Dockerfile is central, serving as a declarative build recipe that automates and standardizes image construction. However, the widespread use of single-stage Dockerfiles, which combine build and runtime in one stage, often yields unnecessarily large images by bundling compilers, development libraries, testing tools, and intermediate artifacts that are not needed in the final runtime environment [13].

*Corresponding author.

Authors' Contact Information: [Dongjin Chen](#), College of Computer Science and Technology, Nanjing University of Aeronautics and Astronautics, Nanjing, China, sz2416168@nuaa.edu.cn; [Wenhua Yang](#), College of Computer Science and Technology, Nanjing University of Aeronautics and Astronautics, Nanjing, China, ywh@nuaa.edu.cn; [Minxue Pan](#), State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China, mxp@nju.edu.cn; [Yu Zhou](#), College of Computer Science and Technology, Nanjing University of Aeronautics and Astronautics, Nanjing, China, zhouyu@nuaa.edu.cn.



This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.

© 2026 Copyright held by the owner/author(s).

ACM 2994-970X/2026/7-ARTFSE070

<https://doi.org/10.1145/3797098>

These oversized images consume more storage and slow deployments due to higher bandwidth needs, while expanding the attack surface by bundling unnecessary, potentially vulnerable components [25]. Recent work also shows that compromised containers can be leveraged as entry points for sophisticated attacks [16, 31]. To mitigate these risks, Docker introduced multi-stage builds [4], which separate build and production stages within a single Dockerfile: build tools and dependencies remain in the builder stage, and only required runtime artifacts are copied into the final image. This design reduces image size and improves security by excluding superfluous components.

While the advantages of multi-stage builds are well recognized and frequently promoted as industry best practices, recent empirical studies find that only a small fraction of Dockerfiles contain multiple `FROM` instructions (the directive used to initialize a new stage), indicating their limited adoption in practice [7, 10]. Developers continue to rely on single-stage Dockerfiles, leaving significant room for optimization. A potential reason for this underutilization lies in the manual effort of refactoring single-stage Dockerfiles into efficient multi-stage ones, which requires understanding the build lifecycle, separating build-time from runtime dependencies, and coordinating artifact transfers across stages via `COPY` (the instruction responsible for copying files from the host context or between previous build stages). This complexity discourages adoption and sustains a gap between best practices and real-world behavior, motivating automated support to convert single-stage Dockerfiles into optimized multi-stage versions.

However, existing research has not specifically addressed the problem of transforming single-stage Dockerfiles into optimized multi-stage versions. Prior work on Dockerfiles has primarily focused on smell understanding [23], rule-based analysis [37], smell fixing [24], and empirical studies of Dockerfile usage patterns [7, 34]. While these efforts provide valuable insights into common issues and developer behaviors, they do not offer concrete solutions for structural optimization. Some recent studies have explored Dockerfile optimization. For example, PARFUM [5, 6] effectively fixes local, syntax-level smells using deterministic rules. However, its rule-based nature limits its ability to capture the broader build logic necessary for multi-stage transformations. Ksontini et al. [15] investigated the use of large language models (e.g., GPT-4o) for automated Dockerfile refactoring, demonstrating promising results in reducing image size, improving build time, and enhancing maintainability. Nonetheless, such approaches may fall short in addressing the deeper structural changes required for robust multi-stage conversion, which demand a holistic and context-aware understanding of build and runtime semantics.

To bridge this gap, we present STAGECRAFT, an automated approach that refactors single-stage Dockerfiles into optimized multi-stage builds. STAGECRAFT proceeds in three steps. First, it statically analyzes the Dockerfile to identify the language and stack, infer architecture and configuration traits, and classify dependencies as build-time or runtime. Second, it applies a lightweight gate that estimates refactoring benefit via a composite score over image bloat, structural inefficiency, and security risk derived from language, dependency, and architecture signals, proceeding only when warranted. Third, it synthesizes a two-stage builder-to-runtime Dockerfile by isolating build tools, copying only required artifacts, and applying production hardening. The result is a multi-stage Dockerfile that reduces image size and attack surface, accelerates iterative builds, and keeps developers in control of final adjustments. In an evaluation on 521 real-world single-stage projects, STAGECRAFT produced multi-stage Dockerfiles that built successfully for 60.3% of targets, and the resulting images were on average 52.2% smaller and contained 50.0% fewer high-risk vulnerabilities than the originals, outperforming baseline methods. In summary, this paper makes the following contributions:

- **Technique:** We introduce STAGECRAFT, an automated approach for converting single-stage Dockerfiles into optimized multi-stage builds.

- **Evaluation:** We conduct a study on 521 real-world projects, benchmarking STAGECRAFT against baselines and showing consistent gains in image quality and build performance while validating the approach’s reliability.
- **Artifacts:** We release STAGECRAFT, the approach’s knowledge assets, and the evaluation dataset to support reproducibility and future research¹.

2 Motivating Example

This section uses an example Dockerfile to motivate the need for multi-stage transformation. Listing 1 presents the Dockerfile. The Dockerfile adopts a conventional single-stage structure, in which all tasks, ranging from application build to runtime environment setup, are performed within a single stage. As a result, it exhibits several common optimization opportunities typically found in Python-based web applications. First, its base image `python:3.8` is a bit bloated and can be further streamlined. Second, it installs heavy build tools such as `build-essential` and `git` (Lines 4–7), which are only needed during build time but remain in the final image. Our profiling shows that these tools alone increase the image size by approximately 100–150 MB. Third, the `curl` package (Line 5), used solely for health checks during the build process, is retained in the runtime image, unnecessarily increasing its footprint. Fourth, `software-properties-common` (Line 6), a utility package for managing repositories and personal package archives, introduces extra runtime complexity without contributing to the application’s execution. Finally, the installation of Python packages may trigger the inclusion of build-time dependencies required for compiling wheels, which are no longer needed after installation but persist in the final image.

More broadly, such patterns are prevalent across many single-stage Dockerfiles. They typically include build tools (e.g., `build-essential` and `python3-dev`), development dependencies (e.g., `devDependencies` in `package.json` for JavaScript applications), and debugging utilities, all of which are irrelevant to production execution [13]. These artifacts not only inflate image size but also introduce security risks, as unnecessary software components may contain known vulnerabilities. For example, vulnerable versions of `curl` or `git` included solely due to build-stage leftovers have been associated with CVEs such as CVE-2018-1000007 and CVE-2022-29187, potentially exposing the container to exploits even when those tools are not required at runtime.

Multi-stage builds, as recommended in Docker’s official best practices [4], provide an effective strategy to mitigate these issues by separating the build and runtime environments. This approach enables developers to retain only essential runtime artifacts in the final image, significantly reducing both image size and attack surface. Despite these benefits, many real-world projects still rely on single-stage Dockerfiles. Manual refactoring into multi-stage configurations remains non-trivial and requires careful coordination across several dimensions. Specifically, developers must: 1) accurately identify which dependencies and tools are needed only at build time versus those required at runtime, 2) restructure Dockerfile instructions across distinct stages while maintaining semantic correctness, 3) design precise artifact transfer strategies (e.g., determining which files or directories to copy from the builder stage), and 4) select appropriate base images tailored to each stage, such as using a full-featured image for building and a minimal image for production.

These challenges pose practical barriers to the adoption of multi-stage Dockerfiles, contributing to a persistent gap between best practices and how developers commonly write Dockerfiles in practice. To bridge this gap, our work aims to propose an automated approach that transforms single-stage Dockerfiles into optimized multi-stage configurations by analyzing Dockerfile structure and dependency semantics.

¹<https://anonymous.4open.science/r/Supplemental-materials-EE15>

```

1 FROM python:3.8
2 WORKDIR /app
3 RUN apt-get update && apt-get install -y \
4     build-essential \
5     curl \
6     software-properties-common \
7     git \
8     && rm -rf /var/lib/apt/lists/*
9 COPY . /app
10 RUN pip install --no-cache-dir -r requirements.txt
11 EXPOSE 8501
12 HEALTHCHECK CMD curl --fail http://localhost:8501/_stcore/health
13 ENTRYPOINT ["streamlit", "run", "Home.py", "--server.port=8501", "--server.address=0.0.0.0"]

```

Listing 1. Example of a single-stage Dockerfile with optimization opportunities.

3 The STAGECRAFT Approach

To address the complexity of converting single-stage Dockerfiles into multi-stage builds, we propose STAGECRAFT, an automated approach based on a three-phase transformation framework.

3.1 Overview

Figure 1 illustrates the three-phase transformation framework implemented in STAGECRAFT. Starting with a single-stage Dockerfile as input, STAGECRAFT first performs parsing and feature extraction (Phase 1). This phase involves a static analysis to capture essential information—such as language-specific traits, dependency relationships, and architecture information—which inform the subsequent necessity assessment and reconstruction stages. Although Dockerfiles are language-neutral, developers often follow language-specific conventions that affect how dependencies are declared and builds are structured. Accordingly, the system first detects the target programming language of the application to infer relevant toolchains and optimization strategies. It also classifies dependencies as either build-time or runtime, as each requires distinct treatment during transformation. In addition, STAGECRAFT extracts architecture and configuration features from the Dockerfile to support the subsequent necessity assessment and reconstruction. These include, for example, the number of effective instructions and the types and quantities of build tools used.

In Phase 2, STAGECRAFT determines whether multi-stage optimization is warranted for the input Dockerfile. This phase helps prioritize transformation efforts and avoid redundant conversions, particularly in minimal or straightforward cases. A lightweight filter is applied first to quickly exclude Dockerfiles that are clearly unsuitable for optimization, such as those built from minimalist base images like `scratch` or those performing only basic file operations. For the remaining candidates, STAGECRAFT computes a multi-dimensional score estimating the potential benefits of conversion. This score quantifies three dimensions (image bloat, structural inefficiency, and security risk) and highlights cases where large build-time toolchains, mixed build and runtime contexts, or unnecessary packages suggest a high payoff from refactoring.

In Phase 3, STAGECRAFT transforms eligible Dockerfiles into optimized multi-stage versions. It first constructs a dual-stage structure grounded in widely adopted best practices, then systematically migrates dependencies, transfers necessary artifacts between stages, and applies production

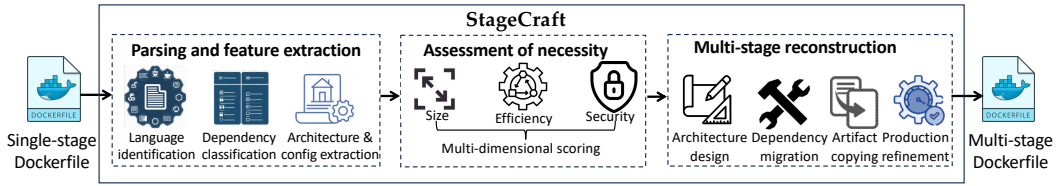


Fig. 1. The Three-Phase Transformation Framework Pipeline.

refinement techniques where appropriate. The resulting Dockerfile is production-ready, offering improved image size, better security, and reduced developer overhead, while maintaining functional compatibility with the original.

3.2 Parsing and Feature Extraction

This phase interprets complex project structures through static analysis of the Dockerfile, extracting features that drive downstream optimization. It combines multi-signal language detection, dependency classification, and architecture/configuration extraction.

3.2.1 Multi-Signal Language Identification. A prerequisite for automated multi-stage Dockerfile reconstruction is accurately identifying the project’s target language and core technology stack. This is crucial because different ecosystems require distinct base images, dependency management strategies, and build artifact structures. Some projects also adopt hybrid stacks. For example, a Dockerfile might include both Python (e.g., `python3-pip`) and Node.js (e.g., `npm`). To ensure a valid transformation, it is necessary to identify the dominant, runtime-relevant stack to select an appropriate build image (e.g., `FROM node:18-alpine`).

To achieve this, STAGECRAFT implements a multi-signal language identification process that aggregates diverse cues embedded in the Dockerfile. It examines the base image, package managers, build tools, framework keywords, configuration files, source file extensions, and runtime directives—signals that, together, provide a strong indication of the target language and underlying stack. For instance, the `FROM` instruction (e.g., `FROM python:3.10-slim`) provides a hint about the target language. Additional signals include package manager commands (e.g., `pip`, `npm`, `mvn`), build tool invocations (e.g., `go build` and `javac`), and framework names (e.g., `gunicorn`, `streamlit`, and `spring-boot`). While all these signals are informative, their significance varies. Base images and runtime directives such as `CMD` and `ENTRYPOINT`, which determine container execution behavior, are typically more decisive than signals like configuration files (`COPY requirements.txt`) or file extensions (e.g., `.py` and `.go`).

STAGECRAFT scans the Dockerfile line by line and keeps a zero-initialized score for each candidate language (e.g., Python and Go). Regular-expression detectors trigger on relevant signals, adding weighted contributions to the score according to estimated importance. Signals and weights are chosen by expert judgment on a linear scale, and details are provided in the supplemental material¹. The language with the highest cumulative score is chosen as the primary target and drives the subsequent multi-stage transformation. Because hybrid stacks (e.g., Python with Node.js) are common, STAGECRAFT also scans for secondary-language signals and performs dependency classification to ensure a complete and correct transformation, as detailed below.

3.2.2 Dependency Classification. This component classifies each declared Dockerfile dependency as either a *build* or *runtime* dependency. Misclassification leads to two opposite risks. A conservative policy may retain unnecessary tools. For example, labeling `gcc` (used only to compile `1.xml`) as

runtime leaves a redundant compiler in the final image, inflating size and increasing the attack surface [25]. Conversely, an aggressive policy may remove required tools. If a service relies on `curl` in a `HEALTHCHECK` instruction but `curl` is treated as build-time and removed, the container can repeatedly fail health checks, disrupting orchestrated deployments such as Kubernetes.

Our empirical analysis shows that dependencies differ in origin, purpose, and runtime relevance. We group them into two broad classes:

- **System-level dependencies:** Installed via system package managers (e.g., `apt-get`); often a mixture of utilities such as `python3` and `git`. Even when listed together, their roles can diverge. For instance, `python3` is commonly needed at runtime, whereas `git` is typically build-only.
- **Application-level dependencies:** Installed via language-specific package managers (e.g., `pip` and `npm`); these libraries usually function as a coherent set serving either runtime behavior or the build process. For example, libraries installed through `pip` usually support either the application's runtime behavior or its build process as a group.

STAGECRAFT applies tailored rules to each class. For *system-level dependencies*, we first match packages against a curated list of known build-time tools (e.g., `gcc`, `make`, and `git`), compiled from official documentation, industry practices, academic studies, and security advisories. The list is provided in our supplementary material¹. Matches are labeled build-time. Otherwise, we assess the package's relation to the detected target language (Section 3.2.1). If it is a core runtime component of that language (e.g., `python3` for Python projects), we classify it as runtime. For remaining cases, STAGECRAFT defaults to a conservative build-time label to preserve build correctness. As an additional safeguard, STAGECRAFT checks whether the dependency appears in runtime-relevant Dockerfile instructions (`CMD`, `ENTRYPOINT`, `HEALTHCHECK`). If so, it is promoted to runtime.

For *application-level dependencies*, STAGECRAFT uses an inheritance rule. Because these packages are introduced by a language-specific package manager, their label follows how the package manager itself is used. If the package manager is present at runtime, all packages it installs are treated as runtime. If the package manager is only needed during the build, its packages are labeled build-time. This strategy reflects common practice, where such managers install a set of libraries that support a specific stage of the software lifecycle.

3.2.3 Architecture and Configuration Extraction. After determining the target language and labeling dependencies, STAGECRAFT analyzes the application's runtime architecture and configuration (e.g., user privileges) to guide necessity assessment and image reconstruction. While stack identification reveals build-time tooling, the runtime role—whether a web API, a static site, or a background worker—often remains ambiguous. This distinction is decisive because it dictates which components must ship in the production image. For example, in a project using both Python and Node.js, even if Python is the target language, the system may implement either (i) a `WEB_API` service that requires a full Python runtime, or (ii) a `STATIC_SITE` pipeline that emits static assets via Node.js tooling, in which case Python is unnecessary at runtime. Misclassification at this stage yields either bloated images with unused components or broken containers missing essential dependencies.

To determine the architecture type, STAGECRAFT employs a rule-based classification framework that maps common architectural patterns (e.g., `WEB_API`, `STATIC_SITE`, `BATCH_JOB`) to technical indicators found in the Dockerfile. These indicators are drawn from four key sources. First, STAGECRAFT inspects **framework- and runtime-specific keywords** such as `fastapi`, `unicorn`, `vite`, or `next.js`, which often signal the application's execution model. Second, it examines **configuration files** referenced or copied in the Dockerfile, such as `nginx.conf`, `webpack.config.js`, or `pom.xml`, which suggest the presence of supporting infrastructure aligned with specific deployment patterns. Third, it analyzes **Dockerfile-specific instructions**,

notably the `EXPOSE` directive, which is commonly associated with web-facing services and thus indicative of a `WEB_API` architecture. Fourth, it considers **build and execution commands** specified via `RUN`, `CMD`, or `ENTRYPOINT`. For example, `CMD["uvicorn", "main:app"]` strongly indicates a Python-based API service, while the sequence `RUN npm run build` followed by `COPY--from=.../app/build/usr/share/nginx/html` typically reflects a static site deployment. Based on these indicators, `STAGECRAFT` scores each candidate architecture type and selects the most likely one. This process follows a similar approach to that used for identifying the target language.

Essential configuration data is also extracted, including the complete base image name and tag from the `FROM` instruction, all environment variables defined via `ENV`, the final working directory specified by `WORKDIR`, and any exposed ports declared through `EXPOSE`. In addition, the exact source and destination paths of each `COPY` instruction are preserved.

3.3 Assessment of Necessity

Based on the parsed results from the previous phase, this stage assesses whether transforming the Dockerfile into a multi-stage build is worthwhile. The goal is to prioritize cases where meaningful gains, such as reduced image size, improved security, or clearer structure, are likely. While multi-stage builds are a best practice, not all Dockerfiles justify the added complexity. Some are too simple, and others offer limited benefits. To avoid unnecessary refactoring, this phase quantitatively estimates optimization potential using extracted language features, dependency profiles, and architectural signals. Only Dockerfiles with sufficient potential proceed to the reconstruction phase.

`STAGECRAFT` employs a multi-dimensional scoring model to quantify the optimization potential of a Dockerfile. The core idea is to convert fixable bad smells—resolvable through multi-stage builds—into a unified score that reflects the value of automated refactoring. The model considers three key dimensions: **Image Bloat**, **Structural Inefficiency**, and **Security Risk**. The final score, S , is computed as:

$$S = w_b \cdot S_{\text{bloat}} + w_e \cdot S_{\text{efficiency}} + w_s \cdot S_{\text{security}} \quad (1)$$

where S_{bloat} , $S_{\text{efficiency}}$, and S_{security} denote the individual scores for each dimension, and w_b , w_e , and w_s are their corresponding weights. By default, all weights are set to 1, enabling equal emphasis across dimensions and allowing adjustments based on specific optimization goals (e.g., prioritizing security over image size).

Image Bloat. This metric captures redundant dependencies in the Dockerfile and is most directly tied to potential image size reduction. The score S_{bloat} is derived from the dependency classification results: $S_{\text{bloat}} = (c_1 \cdot N_{\text{tools}}) + (c_2 \cdot I_{\text{dev}}) + (c_3 \cdot I_{\text{env}})$. Each term reflects a typical source of redundancy: N_{tools} denotes unnecessary tools, I_{dev} indicates excessive development-only dependencies, and I_{env} captures inflated environment setup instructions. The coefficients c_1 , c_2 , and c_3 control their relative contributions.

System-level Build Tools (N_{tools} , $c_1 = 3$): This is the most common redundancy. The system counts system-level dependencies marked as build-time only (e.g., `git`, `make`, `gcc`), assigning 3 points each to the optimization score. This configurable weight serves as a reference for comparing other dimensions, with these dependencies forming the baseline opportunities for optimization.

High-Value Development Libraries (I_{dev} , $c_2 = 10$): This category captures high-impact optimizations. If any development packages ending in `-dev` or `-devel` (e.g., `python3-dev`) appear in system-level build dependencies, the indicator I_{dev} is set to 1, adding a fixed score of 10. These packages contain headers and static libraries unnecessary at runtime, making their removal safe, effective, and weighted three times the baseline.

Redundant Build Environments ($I_{\text{env}}, c_3 = 15$): This dimension covers cases where an entire language runtime is included only for building. If the language identification step (Section 3.2.1) detects a secondary environment used solely at build time (e.g., a Node.js stack for `npm run build`), the indicator I_{env} is set to 1, adding a fixed score of 15. Removing such environments can significantly reduce image size and complexity, justifying the higher weight.

Structural Inefficiency. Beyond image size, build efficiency is also critical for developers. This metric captures the effectiveness of Docker’s layer caching, which is largely influenced by the ordering of instructions in the Dockerfile. Following the widely recommended practice of placing stable dependency installation steps before frequently changing source code (i.e., “stable dependencies before volatile code” [4]) can significantly improve iterative build performance. In addition, recent studies [38] identify several common optimization patterns consistent with this principle, such as placing ENV declarations early and deferring volatile instructions (e.g., RUN or broad COPY commands) to later stages to reduce cache invalidation. These practices are well-defined and can be systematically evaluated. STAGECRAFT assesses adherence to these patterns using an *ideal model comparison* mechanism. Starting from a perfect score of 100, the system deducts points for detected anti-patterns in instruction ordering—such as placing a COPY before a RUN pip install—and computes an efficiency score $E \in [0, 100]$. Dockerfiles with $E < T_e$ (default threshold $T_e = 70$) are considered to have meaningful optimization potential. The corresponding structural efficiency score is calculated as:

$$S_{\text{efficiency}} = \max\left(0, \frac{T_e - E}{d_e}\right) \quad (2)$$

where $d_e = 2$ controls the granularity of the score. This design prevents over-penalizing well-structured Dockerfiles (e.g., $E = 80$) while highlighting inefficient instruction orderings that hurt cache performance. Both T_e and d_e are configurable to match optimization preferences.

Security Risk. This metric quantifies security risks that can be mitigated through multi-stage refactoring. Based on a set of predefined rules drawn from official Docker security guidelines and best practices implemented in widely adopted analysis tools [3, 4], STAGECRAFT conducts a series of checks, including privilege analysis (e.g., use of USER root or implicit root), detection of hard-coded secrets (e.g., credentials in ENV or ARG), and identification of insecure instructions. The security score is computed as: $S_{\text{security}} = \sum_{i=1}^k (w_i \cdot n_i)$, where k is the number of risk categories, n_i is the number of occurrences of the i -th risk, and w_i is its associated weight reflecting severity. Many of these risks can be addressed through multi-stage builds, for instance, by using secrets only in intermediate stages and excluding them from the final image. The rules used for security risk detection, the corresponding risk categories, and their default weights w_i (typically set to 10), along with the instruction ordering patterns for evaluating build efficiency, are provided in our supplementary online materials.

Finally, STAGECRAFT compares the total score S with a configurable threshold Θ (default: 30). A Dockerfile advances to reconstruction only if $S \geq \Theta$. Otherwise, the optimization process terminates. The default, derived from pilot studies and practice, balances transformation effort and benefit, with Section 4 analyzing its impact.

3.4 Multi-stage Reconstruction

If the input Dockerfile passes the assessment in the previous phase, STAGECRAFT proceeds to transform it into an optimized multi-stage version. The reconstruction process is detailed below.

3.4.1 Architecture Design and Base Image Selection. The reconstruction process begins by determining the overall structure of the Dockerfile, which in turn guides the selection of appropriate

base images. A multi-stage design separates the *builder* and *runtime* stages, each optimized for different goals. The builder stage emphasizes reliability, using a fully featured base image (e.g., `python:3.10,node:18`) to provide compilers, libraries, and toolchains. This stage yields a complete environment that includes all dependencies and compiled artifacts, which are then selectively migrated to the subsequent stage.

The runtime stage focuses on minimizing image size and enhancing security. The choice of base image for this stage is guided by the results of the previous two phases. Specifically, STAGECRAFT considers the language and architecture identified in the first phase, as well as the optimization potential scores computed in the second. For example, a high S_{bloat} score or the presence of a compiled language such as Go would prompt the selection of a more aggressive base image, such as a `distroless` image [8], to maximize size reduction. In contrast, for interpreted languages like Python, which may require certain system libraries at runtime, STAGECRAFT adopts a more conservative strategy and chooses an official `slim` variant. This approach removes build-time toolchains while retaining a stable set of common libraries, striking a balance between image minimization and runtime reliability. Additionally, STAGECRAFT adjusts the base image selection based on application-specific architectural signals identified in the first phase (e.g., `AI_ML_CUDA`). In such cases, it selects a customized image that includes essential hardware drivers (e.g., CUDA) but excludes unnecessary toolchains. This targeted strategy ensures substantial image size reduction while preserving the functional integrity of the original environment.

3.4.2 Dependency and Artifact Migration. Once the minimal runtime stage is set, the required dependencies and artifacts are migrated from the builder stage, guided by the following strategies.

Pre-filtering the Build Context. The effectiveness of `COPY` instructions depends on a properly configured `.dockerignore` file, which excludes irrelevant files and directories from the build context [4]. STAGECRAFT automates the generation of this file by composing a dynamic rule set that includes: (1) general exclusion rules (e.g., `.git`, `.vscode`); (2) language-specific rules identified in Phase 1 (e.g., `__pycache__` for Python); and (3) exclusions derived from dependency classification (e.g., source directories of build-only components). This yields a clean context, improves cache efficiency, and avoids including non-runtime content.

Selective Re-installation of System-level Dependencies. While the builder stage includes all tools needed for compilation and packaging, the runtime stage installs only those system packages strictly required for execution. Based on the Phase 1 dependency classification, STAGECRAFT constructs a minimal package installation command (e.g., `RUN apt-get install --no-install-recommends -y . . .`) that includes only packages marked as runtime-essential (e.g., `curl`). Build-only dependencies are excluded to reduce image size and attack surface.

Migration of Application-level Dependencies. For application-level dependencies, STAGECRAFT adopts a *build-and-copy* strategy. Dependencies are first installed in the builder stage using the native package manager (e.g., `pip`), resulting in a complete dependency directory (e.g., `site-packages`). This directory is then copied to the runtime stage via a `COPY --from=builder` instruction. This approach eliminates the need to include the package manager in the final image, ensures consistency between build-time and runtime environments, and avoids dependency reinstallation. It also directly addresses the S_{bloat} metric from the assessment phase by preventing unnecessary development dependencies from being bundled into the production image—an issue commonly associated with Dockerfile anti-patterns.

Selective Copying of Application Artifacts. To migrate build artifacts, STAGECRAFT employs a hybrid strategy that balances optimization with functional completeness. It supports two modes: precise selective copying and a conservative full-copy fallback. By default, STAGECRAFT attempts selective copying, guided by an internal knowledge base that maps technology stacks to their

Table 1. Representative Build Artifact Paths for Common Technology Stacks

Language/Framework	Default Artifact Path	Rationale / Reference
Go	./<binary_name>	Go compiler's default output location [29]
Java (Maven)	target/*.jar	Maven Standard Directory Layout [27]
Java (Gradle)	build/libs/*.jar	Gradle Java Plugin convention [9]
Node.js (React)	build/	Default output of Create React App [19]
Node.js (Vue.js)	dist/	Default output of Vue CLI [30]
Node.js (Angular)	dist/<project-name>/	Default output of Angular CLI

standard build output paths. The knowledge base is built from official documentation and common conventions; a full version is available in the supplementary materials. Table 1 summarizes representative rules for popular frameworks and their corresponding standards. For example, in Go projects, STAGECRAFT locates the final binary based on the compiler's default output path [29], while for Java projects, it follows Maven's standard directory layout [27] or Gradle plugin conventions [9]. These rules enable STAGECRAFT to generate targeted COPY instructions that migrate only the necessary artifacts. If precise paths cannot be identified, or for interpreted languages needing full source code (e.g., Python web apps), STAGECRAFT falls back to copying the entire working directory, trading some optimization for correctness.

3.4.3 Production Refinement. Finally, STAGECRAFT applies a series of optimization and refinement steps to convert the functional Dockerfile into a production-ready artifact. Each step directly addresses a corresponding metric quantified in the assessment phase.

Build Efficiency Optimization. To address build inefficiencies indicated by the $S_{\text{efficiency}}$ score, STAGECRAFT reorders instructions to improve layer cache reuse. Specifically, it enforces the “stable dependencies first” principle [4, 38], ensuring that costly dependency installation steps are only re-executed when relevant changes occur, thereby accelerating iterative development.

Security Hardening. To mitigate security risks captured by the S_{security} score, STAGECRAFT applies several hardening measures. To reduce privilege-related risks, it enforces the principle of least privilege by creating and switching to a non-root user [4]. To eliminate build-time secrets from the final image, sensitive values (e.g., ARG SECRET_KEY) are confined to the builder stage. For compiled languages like Go, STAGECRAFT further reduces the attack surface by stripping debug symbols via build flags such as `-ldflags="-s -w"`.

Production Readiness Enhancements. To improve runtime performance and robustness, STAGECRAFT injects ecosystem-specific configurations identified in Phase 1. For instance, it adds `ENV NODE_ENV=production` for Node.js projects to enable production-mode optimizations [28], and sets `ASPNETCORE_ENVIRONMENT=Production` for ASP.NET Core applications. Additionally, STAGECRAFT inserts a HEALTHCHECK instruction based on the application type (e.g., web service) to enhance reliability in deployment environments.

4 Evaluation

We empirically evaluate STAGECRAFT's effectiveness in refactoring single-stage Dockerfiles into optimized multi-stage configurations, guided by the following research questions:

- **RQ1:** How effective is STAGECRAFT in reconstructing multi-stage Dockerfiles compared to baseline alternatives? Specifically, how does the quality of the reconstructed Dockerfiles, in terms of best practice adherence, structural clarity, and maintainability, compare with the original and baseline-generated versions?

- **RQ2:** How do the container images generated from STAGECRAFT-reconstructed Dockerfiles compare to those produced from the original and baseline-generated versions in terms of build success rate, build efficiency, image size, and security?
- **RQ3:** How accurate are the analysis components in Phase 1 of STAGECRAFT, including target language identification, dependency classification, and architecture extraction? Additionally, how sensitive is the Phase 2 optimization decision process to different parameter settings?

4.1 Experimental Setup

4.1.1 Experimental subjects. We collected a diverse corpus of single-stage Dockerfiles from public GitHub repositories across multiple languages and domains. To cover a range of repository quality, we stratified by star count and randomly sampled 300 repositories with 0 to 200 stars, 300 with 200 to 500 stars, and 400 with 500 to 1,000 stars, each containing a Dockerfile. Following prior work [12, 35], and to avoid ambiguity about build context when multiple Dockerfiles are present, we included only repositories with a Dockerfile at the repository root. We cloned all repositories and built the latest Dockerfile revision, excluding 479 projects with failed builds due to missing files or outdated dependencies. The final dataset includes 521 successful builds spanning Go (158), Python (147), Java (90), Node.js (86), and mixed technologies (40), reflecting real-world diversity. The curated dataset with project lists and URLs is publicly available¹.

4.1.2 Baselines. Since STAGECRAFT is an approach specifically designed to automate the refactoring of single-stage Dockerfiles into optimized multi-stage versions, no existing tools share the same scope and objective. To enable a meaningful comparison, we select two representative baselines that, while not targeting the exact task, share similar goals or can be adapted for this purpose. The first baseline is GPT-4, motivated by the recent success of large language models (LLMs) in tasks such as code refactoring and synthesis. Given each single-stage Dockerfile, we prompt the model to produce a multi-stage refactoring that preserves build behavior and project semantics, keeps the base image family, pins versions when possible, separates builder and runtime stages, and copies only build artifacts. After generation, we attempt a clean build; on failure, we append the full build log to the prompt and request a revision, for up to three rounds. If it still fails, we stop and record the failure. We fix the model and decoding parameters (model=gpt-4.1, temperature=0.2, top_p=1.0, max_tokens=4096, presence_penalty=0, frequency_penalty=0, context_window=200Ktokens). The second is PARFUM [5, 6], a tool that detects and repairs Dockerfile smells via rule-based transformations. While not designed for multi-stage refactoring, its focus on structural improvements and best practices partially aligns with STAGECRAFT's goals, enabling us to assess the added value of STAGECRAFT beyond general smell-oriented optimizations.

4.1.3 Experimental Environment. All experiments ran on Ubuntu 20.04.6 LTS with Docker Engine 28.1.1 (multi-stage supported) and BuildKit enabled. First builds were executed in a clean state with caches pruned; cached builds were measured immediately after a successful first build with identical contexts and sources. To prevent cache interference, projects were built sequentially, and build time was measured end-to-end at the host level. Image vulnerabilities were scanned using Trivy (aquasec/trivy:latest) via `docker run`. Transient network errors during database downloads were retried; persistent failures led to excluding those cases from security-only aggregates.

4.2 Experimental Procedure

4.2.1 RQ1. To address RQ1, we evaluated the ability of STAGECRAFT, GPT-4, and PARFUM to reconstruct and optimize Dockerfiles. We selected 521 single-stage Dockerfiles and applied each method to generate optimized and multi-stage versions. The resulting Dockerfiles were assessed

across three dimensions. We first assessed adherence to Docker best practices with Hadolint [3], a static analysis tool that flags efficiency, security, and image-size issues. The number of reported issues was used as a proxy for compliance. Second, we assessed structural clarity, reflecting how clearly and logically the Dockerfile represents the build process. Poor structure, such as disordered commands, redundant logic, or unclear separation between build and runtime, can hinder understanding and maintenance. Third, we evaluated maintainability, or the ease of updating and debugging Dockerfiles as project requirements evolve. Maintainability is critical in collaborative and CI/CD workflows, where frequent changes are common. Issues such as hardcoded paths, inconsistent formatting, and tightly coupled steps can significantly reduce maintainability.

Since structural clarity and maintainability are not easily captured by automated tools, we conducted a manual evaluation. We randomly sampled 20 Dockerfiles and their transformed versions from each method. We recruited 10 professional developers, each with over five years of Dockerfile experience, to perform a blind review. For each Dockerfile, participants were shown the original and the three anonymized, shuffled transformed versions, and asked to rate each one on a 5-point Likert scale for both structural clarity and maintainability. To ensure consistency, we provided concrete evaluation criteria, including logical command ordering, separation of concerns, avoidance of redundancy, use of clean layering, and overall readability. Final scores were computed as the average across all raters.

4.2.2 RQ2. While RQ1 examines Dockerfile quality, RQ2 evaluates their practical impact through the images they produce. Since Dockerfiles are meant to generate runnable images, this offers a holistic quality measure. For each of the 521 projects, we built images from the original Dockerfiles and those reconstructed by STAGECRAFT, GPT-4, and PARFUM, then compared them across four key dimensions.

First, we measured the build success rate, as a valid build is a prerequisite for any optimization. We recorded whether each Dockerfile could be built without errors in a clean build environment. Failure cases (e.g., missing dependencies or syntax issues) were excluded from further evaluation. For successfully built images, we further compared their image size, a critical factor for deployment efficiency and surface area reduction. To assess image security, we used the Trivy vulnerability scanner to detect known vulnerabilities in installed packages and system libraries. We then measured both first-build time and cached-build time: the former reflects full dependency resolution and downloads, whereas the latter reflects incremental CI/CD runs that benefit from layer reuse. Finally, we recorded image layer counts, since layer granularity affects cache reuse, push/pull performance, and the effective storage and transfer footprint.

4.2.3 RQ3. To assess the reliability of STAGECRAFT's Phase 1 analysis, we conducted a manual validation of its key outputs, which serve as the foundation for subsequent transformations. Phase 1 focuses on three main tasks: target language identification, dependency classification, and architecture extraction. Since each of these tasks produces interpretable outputs, they can be manually validated. For language identification and architecture extraction, which each yield a single result per Dockerfile, we evaluated STAGECRAFT's outputs against manually determined ground truth across the entire dataset. For dependency classification, which involves a larger number of items per file, we randomly sampled 20 Dockerfiles and manually labeled the corresponding 90 dependencies. Three experienced Docker developers (each with over five years of experience) independently performed the labeling for each task. In cases of disagreement, additional reviewers were consulted to reach a consensus. We then compared STAGECRAFT's outputs against the agreed-upon manual annotations to measure accuracy.

Table 2. Comparison of Hadolint Best Practice Violations for Original and Optimized Dockerfiles.

Best Practices	Original	PARFUM	GPT-4	STAGECRAFT
useCopyInsteadOfAdd	36	36 (0.0%)	43 (+19.4%)	0 (-100.0%)
pinGradleVersions	5	5 (0.0%)	4 (-20.0%)	0 (-100.0%)
avoidUnnecessaryShell	6	0 (-100.0%)	0 (-100.0%)	0 (-100.0%)
avoidMaintainer	45	45 (0.0%)	4 (-91.1%)	0 (-100.0%)
mergeRunCommands	7	6 (-14.3%)	2 (-71.4%)	0 (-100.0%)
aptCleanLists	50	9 (-82.0%)	4 (-92.0%)	2 (-96.0%)
useFindXargsNull	5	5 (0.0%)	3 (-40.0%)	1 (-80.0%)
setShell	17	16 (-5.9%)	11 (-35.3%)	5 (-70.6%)
pinNpmVersions	15	15 (0.0%)	14 (-6.7%)	5 (-66.7%)
avoidDnfCleanAll	34	35 (+2.9%)	30 (-11.8%)	12 (-64.7%)
aptUseNoInstallRecommends	65	0 (-100.0%)	61 (-6.2%)	34 (-47.7%)
useWORKDIR	42	39 (-7.1%)	23 (-45.2%)	23 (-45.2%)
addHealthcheck	60	60 (0.0%)	19 (-68.3%)	36 (-40.0%)
useCopyFrom	134	1 (-99.3%)	100 (-25.4%)	89 (-33.6%)
pinPipVersions	75	75 (0.0%)	67 (-10.7%)	51 (-32.0%)
pinGitCommit	27	26 (-3.7%)	28 (+3.7%)	19 (-29.6%)
apkUseNoCache	49	7 (-85.7%)	40 (-18.4%)	36 (-26.5%)
pipUseNoCacheDir	129	2 (-98.4%)	112 (-13.2%)	100 (-22.5%)
pinAptVersions	85	103 (+21.2%)	79 (-7.1%)	68 (-20.0%)
useAbsoluteWORKDIR	8	8 (0.0%)	4 (-50.0%)	7 (-12.5%)
avoidAptAutoremove	25	0 (-100.0%)	14 (-44.0%)	22 (-12.0%)
cleanupUnusedBuildStages	287	221 (-23.0%)	150 (-47.7%)	274 (-4.5%)
avoidLatestTag	38	38 (0.0%)	153 (+302.6%)	180 (+373.7%)

To evaluate the sensitivity of STAGECRAFT’s Phase 2 optimization decisions to parameter settings, we varied key configuration values and observed their impact on reconstruction outcomes. The parameters include: optimization threshold Θ ; weights c_1, c_2, c_3 for the bloatedness score S_{bloat} ; threshold T_e and decay factor d_e for efficiency score $S_{\text{efficiency}}$; and weight w_i for security score S_{security} . While originally selected via empirical tuning and expert input, the effect of these parameters on decisions remains an open question. The default configuration uses $\Theta = 30$, $(c_1, c_2, c_3) = (3, 10, 15)$, $(T_e, d_e) = (70, 2)$, and $w_i = 10$. To assess sensitivity, we varied one parameter at a time while holding others constant and checked whether the reconstruction decision for each Dockerfile changed. Specifically, we tested: $\Theta \in \{20, 40, 50\}$; $(c_1, c_2, c_3) \in \{(3, 3, 3), (10, 10, 10), (5, 15, 20)\}$; $(T_e, d_e) \in \{(60, 2), (70, 5), (80, 2)\}$; and $w_i \in \{5, 15\}$. We applied each variant across evaluation Dockerfiles and compared decisions with the default. This analysis reveals the robustness of STAGECRAFT’s decision process to parameter changes.

4.3 Experimental Results

4.3.1 RQ1: Dockerfile Transformation Effectiveness. We applied STAGECRAFT, GPT-4, and PARFUM to reconstruct the 521 Dockerfiles in our dataset, then evaluated adherence to Docker best practices using Hadolint. Table 2 reports the comparison between STAGECRAFT and the baselines. Hadolint checks a wide range of practices. For example, “useCopyInsteadOfAdd” targets clarity and layer control, “pinNpmversions” aims at reproducibility and security, “avoidLatestTag” reduces upgrade risks, and “useWORKDIR” improves readability and correctness. Across the 521 Dockerfiles, we observed 49 kinds of distinct best practice rule violations with varying frequencies. Due to space limits, Table 2 reports only rules with at least five occurrences of violations, and the complete results are provided in the online supplementary material¹.

Table 2’s second column shows Hadolint results for the original Dockerfiles, and columns 3–5 show the results for the variants produced by PARFUM, GPT-4, and STAGECRAFT, respectively. Each cell shows the number of detected violations, the relative change with respect to the original, and a compact bar indicating the magnitude and direction of change. For deterministic, low-ambiguity rules such as “useCopyInsteadOfAdd”, “avoidMaintainer”, and “avoidUnnecessaryShell”, STAGECRAFT eliminated all violations (100%), outperforming both PARFUM and GPT-4. In build-efficiency and caching-related rules, STAGECRAFT also achieved substantial reductions, e.g., “aptCleanLists” (96.0%), “mergeRunCommands” (100%), and “useFindXargsNull” (80.0%). There are, however, rules where PARFUM removes more violations than STAGECRAFT and GPT-4. This reflects PARFUM’s rule-driven approach, with limited consideration of context and downstream effects. For instance, enforcing “aptUseNoInstallRecommends” across the board can inadvertently omit packages that are “Recommended” yet functionally required at runtime (e.g., CA certificates, locales, or CLI helpers), leading to latent failures. Likewise, aggressively rewriting to `COPY --from` without precise artifact scoping risks missing dynamically produced paths or altering ownership/permissions, which can break entrypoints or health checks. STAGECRAFT adopts a conservative stance in such cases to preserve build correctness. This design trade-off is more visible in “avoidLatestTag”. To maximize compatibility in unknown environments, STAGECRAFT may select a fully featured base image for the builder stage, which can trigger a warning for using `:latest`. GPT-4 often makes a similar choice, whereas PARFUM typically leaves this unchanged. Although this choice increases a lint warning, it yields a stable build context. Developers can subsequently pin an explicit version in the final runtime stage with minimal effort. Overall, STAGECRAFT favors delivering a functionally correct, maintainable multi-stage template over aggressively satisfying every lint rule. Additionally, for context-dependent checks such as “cleanupUnusedBuildStages”, STAGECRAFT proceeds cautiously to avoid removing stages that are conditionally referenced via `ARG` or multi-target builds.

Ten experienced developers rated 20 random original Dockerfiles and their transformed versions on a 5-point Likert scale for clarity and maintainability. Figure 2 shows the rating distributions as split violin plots. STAGECRAFT attains higher medians on both dimensions than the original, PARFUM, and GPT-4. At the same time, its distributions are distinctly bimodal, with mass near 5 and 1. This pattern suggests that STAGECRAFT delivers expert-level outcomes when the input aligns with its assumptions (e.g., correctly inferred target language and architecture, clear separation of build/runtime dependencies), but can degrade when faced with atypical configurations beyond its heuristics. For example, low scores commonly occur with polyglot pipelines (e.g., a Python service with Node.js assets), dynamic build paths that hinder precise artifact copying, or entry points that depend on utilities misclassified as build time (e.g., `curl` in `HEALTHCHECK`), leading to extra stages, overly broad `COPY` rules, or redundant base layers.

By contrast, GPT-4 shows a wide mid-to-low spread. Reviewers frequently flagged patterns that hinder clarity and long-term upkeep, such as collapsing unrelated commands into a single long `RUN` instruction, placing `COPY` too early (which erodes cache locality and blurs dependency boundaries), intermixing build and runtime steps in one stage, or leaving package managers in the final image. PARFUM’s scores form a compact distribution slightly above the originals, consistent with its design: it reliably applies local fixes (e.g., version pinning, merging trivial `RUN` chains) but does not attempt deeper architectural restructuring. Taken together, the full distributions indicate that STAGECRAFT offers higher median quality but with higher variance, reflecting a design choice to prioritize strong improvements on common patterns while avoiding aggressive, potentially brittle rewrites on less standard cases. Beyond the distributional view, we apply paired Wilcoxon signed-rank tests to per-sample medians (10 raters). Results align with Figure 2: STAGECRAFT is significantly higher than the original and PARFUM on both clarity and maintainability; it also significantly outperforms GPT-4 on maintainability and shows a clear advantage on clarity.

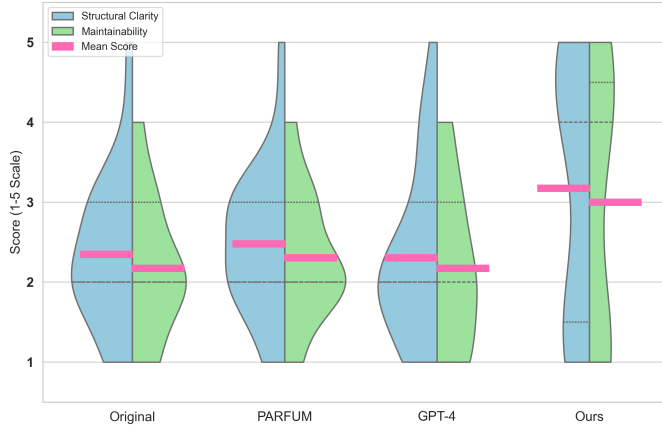


Fig. 2. Split Violin Distributions of Ratings for Structural Clarity and Maintainability.

4.3.2 RQ2: Image Quality Evaluation. This RQ evaluates the impact of transformed Dockerfiles on the resulting images. Figure 3 summarizes the results. All original Dockerfiles built successfully (100%) since the dataset was pre-filtered. Notably, both STAGECRAFT and GPT-4 succeeded in generating syntactically valid multi-stage structures (containing more than one `FROM` instruction) for all projects. However, ensuring these artifacts build successfully proved more challenging. After transformation, all three methods had lower success rates, highlighting the challenge of fully automating architectural refactoring. PARFUM exhibits the smallest decline, as its edits are largely local and avoid deep changes to dependencies or architecture. STAGECRAFT achieves a 60.3% automated build success rate, slightly higher than GPT-4’s 57.8%. Manual inspection indicates that failures in STAGECRAFT primarily arise from context-specific nuances, such as path assumptions in shell scripts or environment-specific package names, rather than from the multi-stage structure itself. In contrast, GPT-4’s failures often stemmed from hallucinations, consistent with findings in other code refactoring studies [1, 18, 20], such as generating non-existent image tags or mixing incompatible distribution variants. To better interpret the build success rates, it is crucial to recognize the inherent complexity gap between simple syntax updates and architectural restructuring. While any modification risks build stability, migrating legacy projects to multi-stage builds is particularly non-trivial. PARFUM, which performs only local, syntax-level rewrites without cross-stage restructuring, saw its build success rate drop from 100% to 75.6%. In contrast, STAGECRAFT performs much deeper architectural refactoring yet still achieves a success rate of 60.3%, while delivering superior performance metrics across other dimensions. Even in cases where the automated build fails, the value of STAGECRAFT lies in its role as a developer assistant. It performs the heavy lifting of structural transformation, enabling developers to start from a sound multi-stage template rather than a raw single-stage Dockerfile. By automating the more error-prone refactoring steps, STAGECRAFT reduces the manual burden, reducing the remaining work to merely resolving specific context-dependent issues.

Once a successful build is achieved, the architectural benefits of STAGECRAFT’s transformation become clear across multiple dimensions. The most pronounced gains are in image footprint and security. As shown in Figure 3, STAGECRAFT reduces the average final image size by 52.2% (from 1353MB to 647MB), exceeding PARFUM (17.2%) and GPT-4 (37.2%). Layer count also drops from 7.2 to 3.2 on average. These improvements follow directly from STAGECRAFT’s design: it confines

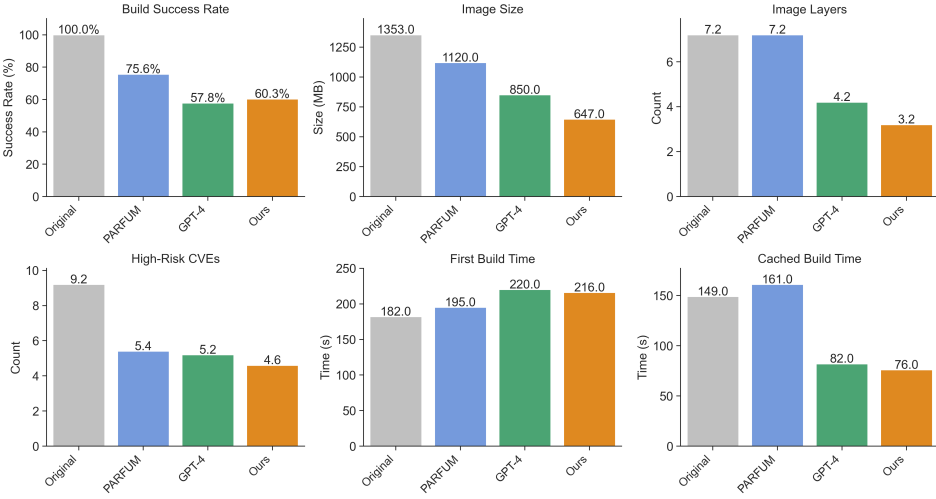


Fig. 3. Image Quality and Build Performance: Original vs. Transformed Dockerfiles.

compilers, package managers, and other build-only tools to the builder stage; narrows artifact transfer to precisely targeted outputs rather than workspace-level copies; and removes transient files. Because builder layers are excluded from the final image, concentrating build activity upstream translates into fewer layers and a smaller runtime footprint. Trivy correspondingly reports a 50.0% reduction in high-risk vulnerabilities (from 9.2 to 4.6), consistent with eliminating build-time components from the production image.

This architectural refactoring also benefits developer productivity. By arranging layers to maximize cache reuse, STAGECRAFT cuts cached build time by 49.0% (from 149s to 76s), which is the best among the evaluated methods. The initial build time increases by 18.7% (from 182s to 216s), reflecting the extra work performed in the builder stage. This upfront cost is typical of multi-stage builds and enables faster subsequent iterations. Overall, the results show that STAGECRAFT automates the central multi-stage trade-off: a modest one-time cost during the first build in exchange for sustained gains in image size, security, and iterative build efficiency, outperforming the baselines on these downstream metrics.

4.3.3 RQ3: Analysis Accuracy and Sensitivity. This RQ evaluates the internal reliability of STAGECRAFT by assessing two aspects of its decision process: the accuracy of Phase 1 analysis and the sensitivity of Phase 2 assessment. For Phase 1, we validated the three core analyses against human annotations. For language and architecture identification, STAGECRAFT achieved 95% accuracy on both tasks. The few errors occurred in mixed-stack projects where prominent build-time signals (e.g., Node.js tooling) masked the true runtime stack (e.g., Go), or in Dockerfiles with weak architectural cues (e.g., web services without an EXPOSE instruction). For dependency classification, we manually verified 90 dependencies sampled from 20 Dockerfiles. STAGECRAFT misclassified 8 of them, yielding an error rate of approximately 8.9%. Errors concentrated in system-level utilities, and application-level dependencies were perfectly classified. In particular, utilities such as `ffmpeg` and `ImageMagick` were occasionally labeled as build-time when they were not on the known-tools list and were not explicitly referenced by `CMD`, `ENTRYPOINT`, or `HEALTHCHECK`. This reflects a design trade-off: in the absence of runtime evidence, STAGECRAFT prefers to minimize the runtime

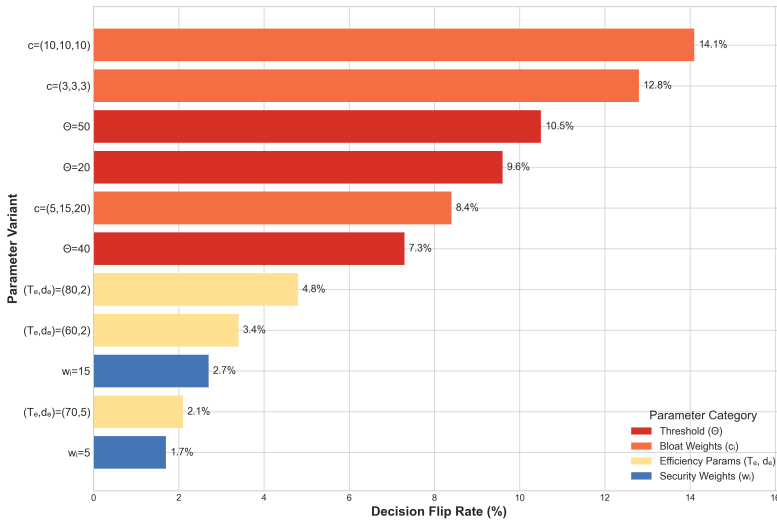


Fig. 4. Tornado Plot of STAGECRAFT's Decision Sensitivity to Scoring Parameters. Bars Show the Share of Dockerfiles Whose Refactor Decision Flips Relative to the Default.

footprint. The risk of omitting a latent runtime utility is mitigated by checking runtime directives, allowing user overrides, and exposing configuration to promote specific packages to runtime.

For Phase 2, we assessed parameter sensitivity by varying the optimization threshold and scoring weights, as described in Section 4.2.3. The tornado plot in Figure 4 reports the decision-flip rate, defined as the fraction of projects whose refactor/not-refactor outcome changes relative to the default configuration. The decision gate is most sensitive to the bloat score weights, which produced the largest shifts (14.1% and 12.8%), indicating that the size-oriented weighting scheme—especially the emphasis on high-impact `-dev` libraries—strongly influences which Dockerfiles are selected for transformation. The overall optimization threshold Θ is the next most influential factor, acting as the final gatekeeper and controlling the tool's trigger rate. In contrast, parameters related to efficiency and security consistently yielded flip rates below 5%, suggesting robustness to these settings on our dataset. Overall, decisions were stable under moderate changes; most differences arose in borderline cases near the threshold, and qualitative trends remained consistent.

4.4 Discussions

Ensuring that refactored multi-stage containers maintain the functional equivalence of the original builds is critical. While guaranteeing formal correctness remains challenging without a complete behavioral specification, STAGECRAFT mitigates the risk of functional regression through three layers of design decisions. First, to prevent the removal of tools implicitly needed at runtime, STAGECRAFT does not rely solely on static package lists but explicitly parses runtime directives and would promote a tool to a runtime dependency if it appears in execution directives. Second, regarding artifact migration, STAGECRAFT uses a hybrid strategy: it attempts precise selective copying only when the project strictly matches a known stack pattern; otherwise, it falls back to a full-copy strategy to ensure no hidden assets are left behind. Third, to avoid breaking dependencies relying on standard libraries, STAGECRAFT avoids aggressive OS swaps, ensuring that standard system libraries required by the language runtime remain available.

The architectural design of STAGECRAFT prioritizes both extensibility and portability across diverse technology stacks. Internally, we adopt a hybrid structure where language-dependent logic, specifically stack identification (Phase 1) and command selection (Phase 3), is confined to isolated components, while the core structural generation logic remains language-agnostic. This decoupling allows the current implementation to support major ecosystems (e.g., Python, Node.js, Java, Go) and facilitates extension to new stacks via configuration updates. Consistent with this lightweight philosophy, we deliberately restrict the input scope to the Dockerfile rather than the full project context to avoid the complexity of parsing heterogeneous build systems. Nonetheless, we acknowledge that taking the whole project context, including source code, build scripts, and manifests, as input could further improve the quality of multi-stage builds, and we consider the exploration of such hybrid analysis strategies a promising direction for future research.

Our findings offer several practical implications for both software practitioners and researchers in the domain of cloud-native development. For practitioners, this work first lowers the technical barrier to adopting multi-stage builds. While manual refactoring is complex and error-prone, automated scaffolding tools can significantly reduce the effort. Even in cases where the automated build fails, the generated Dockerfile provides a valid structural draft that adheres to best practices, serving as a superior starting point compared to writing from scratch. Second, the proposed approach translates to tangible operational efficiency. The observed 52.2% reduction in image size and 49.0% reduction in cached build time directly impact cloud costs, saving bandwidth and storage while accelerating CI/CD feedback loops. Third, the results facilitate automated DevSecOps. By achieving a 50.0% reduction in high-risk vulnerabilities through architectural restructuring, our approach demonstrates that security hardening can be effectively shifted left into the build generation phase. For researchers, unlike prior studies focused on local smell detection, our work demonstrates the feasibility of automating structural refactoring. Effectively addressing image bloat and security requires architectural-level transformations rather than syntax-level fixes, encouraging future research to move beyond local optimizations and tackle complex, dependency-aware architectural changes. Furthermore, our comparison with GPT-4 highlights that domain-specific static analysis remains crucial. General-purpose LLMs struggle with the precise context management required for build systems (e.g., file paths and dependency trees). This suggests that future research should explore hybrid approaches, combining the generative capabilities of LLMs with the precision of static analysis to ensure build correctness.

5 Threats to Validity

We identify several potential threats to the validity of our findings and discuss the steps taken to mitigate them.

Selection bias in the Dockerfile corpus. Our evaluation is based on publicly available Dockerfiles collected from GitHub repositories. These samples may not fully represent proprietary enterprise projects or Dockerfiles used in specialized industrial domains. Additionally, although we included projects spanning multiple languages (e.g., Python, Node.js, Java, and Go), our findings may not generalize to all technology stacks. To reduce this risk, we curated a diverse corpus by sampling projects from various ecosystems, project sizes, and application domains.

Accuracy of static analysis. STAGECRAFT relies on heuristic-based techniques in Phase 1 for target language identification, dependency classification, and architecture extraction. Inaccuracies in these analyzes could impact the effectiveness of subsequent transformations. To mitigate this, we designed the heuristics based on language-specific patterns and best practices and cross-validated the results through manual inspection. Furthermore, we conducted targeted experiments (RQ3) to assess the accuracy of these analyzes, demonstrating their overall reliability.

Parameter sensitivity. Several parameters in STAGECRAFT, particularly in Phase 2's optimization assessment, require manual tuning, such as scoring weights and decision thresholds. Although these values were chosen based on empirical trials and domain knowledge, they may still influence the tool's behavior. To address this concern, we performed a sensitivity analysis to examine the effect of varying these parameters (RQ3). The results show that STAGECRAFT's decisions are relatively robust to moderate parameter changes, and all parameters are configurable for adaptation to different contexts.

Measurement reliability. Our evaluation combines automated and manual assessments, each with potential limitations. For best practice adherence, we use Hadolint, a widely adopted static analyzer. While generally reliable, it may miss certain practices or project-specific nuances. For structural clarity and maintainability, experienced developers rated anonymized and randomized Dockerfiles using a 5-point Likert scale. Each file was independently scored by multiple raters.

6 Related Work

Although Docker introduced multi-stage builds in 2017 [4] and promotes them in official best practices, adoption remains limited. Empirical studies [7, 10, 11] show that only a small fraction of Dockerfiles include multiple `FROM` instructions (a necessary condition for implementing multi-stage builds), indicating a slow uptake of this feature.

Few studies have directly addressed the automated transformation of existing Dockerfiles into multi-stage builds. More broadly, prior work has examined Dockerfile refactoring. Ksontini et al. [14] proposed DRMiner, which identifies refactoring opportunities using an AST-based component-matching algorithm and rule-based heuristics. However, DRMiner focuses on analyzing historical changes (e.g., GitHub commits) and does not support automated transformations. In follow-up work, Ksontini et al. [15] investigated using LLMs (GPT-3.5 and GPT-4) for Dockerfile refactoring. While LLMs handle simple edits well, they struggle with complex tasks like converting single-stage Dockerfiles into maintainable multi-stage versions, often producing inconsistent results and overlooking dependency and architectural constraints. In our evaluation, we included GPT-4 as a baseline and found that accurate multi-stage transformation requires fine-grained reasoning over configuration, dependencies, and runtime behavior, which GPT-4 cannot directly handle for this task. Although research on multi-stage Dockerfile conversion is limited, related work on smell detection, repair, and optimization is more extensive, as discussed below.

Early work on Dockerfile smell detection primarily focused on identifying common anti-patterns. Rosa et al. [23] conducted a systematic empirical evaluation of various types of Dockerfile smells, examining the extent to which the issues flagged by Hadolint align with the perceptions of experienced Dockerfile developers. Wu et al. [33] studied Dockerfile evolution and showed that complexity tends to increase faster than maintainability, underscoring the need for automation. Zhou et al. [37] propose an approach for mining implicit rules and detecting potential violations of such rules in Dockerfiles. Static analysis tools such as Hadolint [3] and Dockerfile Linter [22] enforce best practices through rule-based checks but lack transformation capabilities. Binnig [2] introduced a tool to detect 23 smell types, while Zhang et al. [36] investigated the link between Dockerfile smells and build failures, identifying 19 categories. These studies provide important insights into Dockerfile quality, but they concentrate on smell detection rather than structural remediation, especially the architectural changes required for multi-stage builds.

More recent work has shifted toward automated repair. Henkel et al. [12] introduced SHIP-WRIGHT, a system that identifies and repairs broken Dockerfiles. It leverages a modified BERT model to embed build logs and cluster failures, enabling the design of 13 rule-based repair strategies through a search-guided process. PARFUM [5, 6] applies program transformation to fix five types of Dockerfile smells, but its scope is limited to local edits such as version pinning and command

merging. DockerCleaner [32] uses machine learning to repair Dockerfiles with promising accuracy, yet its focus remains on surface-level issues. Lin et al. [17] examined performance-sensitive Docker configurations, offering insights that informed our design. Overall, existing approaches primarily operate at the syntactic or instruction level, emphasizing best practice adherence and localized optimizations. In contrast, higher-level architectural concerns, such as the decomposition of monolithic Dockerfiles into multi-stage builds, have received limited attention. To the best of our knowledge, our work is the first to explicitly address the automated transformation of single-stage Dockerfiles into multi-stage ones. This transformation requires reasoning about build dependencies, runtime requirements, and role-specific file usage—capabilities not addressed by existing repair tools.

Additional studies have focused on container debloating and image minimization. Tools such as DockerSlim (now Slim.AI) [26] and Cimplifier [21] reduce image size by removing unused runtime components or applying binary-level optimizations. While effective post-build, these techniques operate reactively on the container image rather than improving the Dockerfile itself. Complementary to debloating, Doctor [38] reduces rebuild time by reordering Dockerfile instructions using dependency and edit-history analysis. Doctor optimizes intra-file ordering rather than structural refactoring. Unlike these works, our work proactively transforms Dockerfiles at the source level, enabling automated conversion to multi-stage builds. This transformation not only aligns with best practices but also facilitates downstream benefits such as image slimming and enhanced security through more precise dependency management.

7 Conclusion

This paper presented STAGECRAFT, an approach that converts single-stage Dockerfiles into multi-stage builds through static stack and dependency analysis, a benefit gate, and a synthesis step that isolates build tools, migrates only required artifacts, and applies production hardening. On 521 real-world projects, STAGECRAFT generated working multi-stage Dockerfiles for 60.3% of cases; the resulting images were on average 52.2% smaller and had 50.0% fewer high-risk vulnerabilities, outperforming baselines.

8 Data Availability

We release the tool, curated knowledge assets, and the full evaluation dataset at <https://anonymous.4open.science/r/Supplemental-materials-EE15> to support reproducibility and future research.

Acknowledgment

This work was supported by the National Natural Science Foundation of China (Nos. 62372232, 62372227).

References

- [1] Abhiram Bellur, Fraol Batole, Mohammed Raihan Ullah, Malinda Dilhara, Yaroslav Zharov, Timofey Bryksin, Kai Ishikawa, Haifeng Chen, Masaharu Morimoto, Takeo Hosomi, Tien N. Nguyen, Hridesh Rajan, Nikolaos Tsantalis, and Danny Dig. 2025. Together We are Better: LLM, IDE and Semantic Embedding to Assist Move Method Refactoring. In *2025 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. 1–13. <https://doi.org/10.1109/ICSME64153.2025.00046>
- [2] Christoph Binnig. 2020. *Dockerfile Static Analysis for Security and Best Practices*. Master’s thesis. Technical University of Munich.
- [3] Hadolint Contributors. 2023. Hadolint: Dockerfile linter, validate inline bash, written in Haskell. <https://github.com/hadolint/hadolint>. Accessed: 2025-06-10.
- [4] Docker Inc. 2024. Best practices for writing Dockerfiles. https://docs.docker.com/develop/develop-images/dockerfile_best-practices/. Accessed: 2025-07-16.
- [5] Thomas Durieux. 2023. Parfum: Detection and Automatic Repair of Dockerfile Smells. arXiv:2302.01707 [cs.SE] <https://arxiv.org/abs/2302.01707>

- [6] Thomas Durieux. 2024. Empirical Study of the Docker Smells Impact on the Image Size. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (Lisbon, Portugal) (ICSE '24)*. Association for Computing Machinery, New York, NY, USA, Article 208, 12 pages. <https://doi.org/10.1145/3597503.3639143>
- [7] Calvin Eng and Abram Hindle. 2021. Revisiting Dockerfiles in Open Source Software Over Time. In *2021 IEEE/ACM 18th International Conference on Mining Software Repositories (MSR)*. 449–459. <https://doi.org/10.1109/MSR52588.2021.00057>
- [8] Google. 2024. Distroless Docker Images. <https://github.com/GoogleContainerTools/distroless>. Accessed: 2025-07-10.
- [9] Gradle Inc. 2024. The Java Plugin - Project Layout. https://docs.gradle.org/current/userguide/java_plugin.html#sec:java_project_layout. Accessed: 2025-07-10.
- [10] Jordan Henkel, Christian Bird, Shuvendu K. Lahiri, and Thomas Reps. 2020. A Dataset of Dockerfiles. In *Proceedings of the 17th International Conference on Mining Software Repositories (Seoul, Republic of Korea) (MSR '20)*. Association for Computing Machinery, New York, NY, USA, 528–532. <https://doi.org/10.1145/3379597.3387498>
- [11] Jordan Henkel, Christian Bird, Shuvendu K. Lahiri, and Thomas Reps. 2020. Learning from, understanding, and supporting DevOps artifacts for docker. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering (ICSE '20)*. ACM, 38–49. <https://doi.org/10.1145/3377811.3380406>
- [12] Jordan Henkel, Denini Silva, Leopoldo Teixeira, Marcelo d'Amorim, and Thomas Reps. 2021. Shipwright: A Human-in-the-Loop System for Dockerfile Repair. In *Proceedings of the 43rd International Conference on Software Engineering (Madrid, Spain) (ICSE '21)*. IEEE Press, 1148–1160. <https://doi.org/10.1109/ICSE43902.2021.00106>
- [13] Shahriar Islam et al. 2021. An Empirical Study of Dependency-Related Bloat in Docker Images. In *Proceedings of the 18th International Conference on Mining Software Repositories (MSR)*. IEEE, 174–184.
- [14] Emna Ksontini, Aycha Abid, Rania Khalsi, and Marouane Kessentini. 2024. DRMiner: A Tool For Identifying And Analyzing Refactorings In Dockerfile. In *Proceedings of the 21st International Conference on Mining Software Repositories (Lisbon, Portugal) (MSR '24)*. Association for Computing Machinery, New York, NY, USA, 584–594. <https://doi.org/10.1145/3643991.3644921>
- [15] Emna Ksontini, Meriem Mastouri, Rania Khalsi, and Wael Kessentini. 2025. Refactoring for Dockerfile Quality: A Dive into Developer Practices and Automation Potential. In *2025 IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR)*. 788–800. <https://doi.org/10.1109/MSR66628.2025.00116>
- [16] Haneul Lee, Soonhong Kwon, and Jong-Hyoun Lee. 2023. Experimental analysis of security attacks for Docker Container Communications. *Electronics* 12, 4 (2023), 940. <https://doi.org/10.3390/electronics12040940>
- [17] Jingzhu Lin, Radu Marinescu, Cristian Cadar, and Grigore Rosu. 2021. Understanding and Auto-Adjusting Performance-Sensitive Configurations. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. ACM, 230–242.
- [18] Bo Liu, Yanjie Jiang, Yuxia Zhang, Nan Niu, Guangjie Li, and Hui Liu. 2025. Exploring the potential of general purpose LLMs in automated software refactoring: an empirical study. *Automated Software Engineering* 32, 1 (01 Mar 2025), 26. <https://doi.org/10.1007/s10515-025-00500-0>
- [19] Meta Inc. 2024. Create React App - Production Build. <https://create-react-app.dev/docs/production-build/>. Accessed: 2025-07-10.
- [20] Dorin Pomian, Abhiram Bellur, Malinda Dilhara, Zarina Kurbatova, Egor Bogomolov, Timofey Bryksin, and Danny Dig. 2024. Next-Generation Refactoring: Combining LLM Insights and IDE Capabilities for Extract Method. In *2024 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. 275–287. <https://doi.org/10.1109/ICSME58944.2024.00034>
- [21] Vaibhav Rastogi, Drew Davidson, Lorenzo De Carli, Somesh Jha, and Patrick McDaniel. 2017. Cimplier: Automatically Debloating Containers. In *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering (Paderborn, Germany) (ESEC/FSE 2017)*. Association for Computing Machinery, New York, NY, USA, 476–486. <https://doi.org/10.1145/3106237.3106271>
- [22] Replicatedhq. 2023. dockerfilelint: An opinionated Dockerfile linter. <https://github.com/replicatedhq/dockerfilelint>. Accessed: 2025-06-10.
- [23] Giovanni Rosa, Simone Scalabrino, Gregorio Robles, and Rocco Oliveto. 2024. Not all Dockerfile Smells are the Same: An Empirical Evaluation of Hadolint Writing Practices by Experts. In *2024 IEEE/ACM 21st International Conference on Mining Software Repositories (MSR)*. 231–241.
- [24] Giovanni Rosa, Federico Zappone, Simone Scalabrino, and Rocco Oliveto. 2024. Fixing Dockerfile smells: an empirical study. *Empirical Software Engineering* 29, 108 (Jul 2024). <https://doi.org/10.1007/s10664-024-10471-7>
- [25] Rui Shu, Xiaohui Gu, and William Enck. 2017. A Study of Security Vulnerabilities on Docker Hub (CODASPY '17). Association for Computing Machinery, New York, NY, USA, 269–280. <https://doi.org/10.1145/3029806.3029832>
- [26] Slim.AI. 2023. DockerSlim: Make your containers smaller, more secure and do it without changing anything. <https://github.com/slimtoolkit/slim>. Accessed: 2025-06-10.
- [27] The Apache Maven Project. 2024. Maven - Introduction to the Standard Directory Layout. <https://maven.apache.org/guides/introduction/introduction-to-the-standard-directory-layout.html>. Accessed: 2025-07-10.

- [28] The Express.js Project. 2024. Production Best Practices: Performance and Reliability. <https://expressjs.com/en/advanced/best-practice-performance.html>. Accessed: 2025-07-17.
- [29] The Go Authors. 2024. Command go - The Go Programming Language. https://go.dev/cmd/go/#hdr-Build_and_test_packages_and_dependencies. Accessed: 2025-08-20.
- [30] The Vue.js Team. 2024. Vue.js - Production Deployment. <https://vuejs.org/guide/best-practices/production-deployment.html>. Accessed: 2025-08-20.
- [31] Gogulakrishnan Thiagarajan and Prabhudharshi Nayak. 2025. Docker under Siege: Securing Containers in the Modern Era. *International Journal of Scientific Research in Computer Science, Engineering and Information Technology* 11, 1 (2025), 3674–3719. <https://doi.org/10.32628/CSEIT25112773>
- [32] Yiwen Wu, Peng Liu, Yang Zhang, and Xin Peng. 2023. DockerCleaner: Automatic Repair of Dockerfile Smells. In *Proceedings of the 30th IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 832–843.
- [33] Yiwen Wu, Yang Zhang, Tao Wang, and Huaimin Wang. 2020. Characterizing the Occurrence of Dockerfile Smells in Open-Source Software: An Empirical Study. *IEEE Access* 8 (2020), 34127–34139. <https://doi.org/10.1109/ACCESS.2020.2973750>
- [34] Yiwen Wu, Yang Zhang, Tao Wang, and Huaimin Wang. 2020. Dockerfile Changes in Practice: A Large-Scale Empirical Study of 4,110 Projects on GitHub. In *2020 27th Asia-Pacific Software Engineering Conference (APSEC)*. 247–256. <https://doi.org/10.1109/APSEC51365.2020.00033>
- [35] Yiwen Wu, Yang Zhang, Kele Xu, Tao Wang, and Huaimin Wang. 2023. Understanding and Predicting Docker Build Duration: An Empirical Study of Containerized Workflow of OSS Projects. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (Rochester, MI, USA) (ASE '22)*. Association for Computing Machinery, New York, NY, USA, Article 111, 13 pages. <https://doi.org/10.1145/3551349.3556940>
- [36] Yang Zhang, Gang Yin, Tao Wang, Yue Yu, and Huaimin Wang. 2018. An Insight Into the Impact of Dockerfile Evolutionary Trajectories on Quality and Latency. In *2018 IEEE 42nd Annual Computer Software and Applications Conference (COMPSAC)*. 138–143. <https://doi.org/10.1109/COMPSAC.2018.00026>
- [37] Yu Zhou, Weilin Zhan, Zi Li, Tingting Han, Taolue Chen, and Harald Gall. 2023. DRIVE: Dockerfile Rule Mining and Violation Detection. *ACM Trans. Softw. Eng. Methodol.* 33, 2, Article 30 (Dec. 2023), 23 pages. <https://doi.org/10.1145/3617173>
- [38] Zhiling Zhu, Tieming Chen, Chengwei Liu, Han Liu, Qijie Song, Zhengzi Xu, and Yang Liu. 2025. Doctor: Optimizing Container Rebuild Efficiency by Instruction Re-orchestration. *Proc. ACM Softw. Eng.* 2, ISSTA, Article ISSTA001 (June 2025), 23 pages. <https://doi.org/10.1145/3728870>

Received 2025-09-11; accepted 2025-12-22